

Q&A Cheatsheet — Adaptive Algorithms

Continuous Optimization Project Presentation

Erion Krasniqi & Denis Mustafa Xhabrahi

Golden Rule for oral Q&A: Answer in **3 sentences**: **intuition** → **project connection** → **result / consequence**. Avoid long formulas unless explicitly asked. Start intuitive, then become precise if needed.

1. Big Picture

Q: What is the main goal of your project?

Oral answer: We compare adaptive first-order optimization methods against SGD. We focus on Adagrad, AdaGrad-Norm, RMSprop and Adam. Experimentally, we test whether they improve convergence speed and final performance on A9a and MNIST.

Q: What is your main takeaway?

Oral answer: Adaptive methods converge much faster in early iterations. However, final test accuracy is often very similar across optimizers. So the main benefit is **optimization speed**, not necessarily better final generalization.

Q: Why adaptive step sizes?

Oral answer: A fixed learning rate is difficult because one value may be too large for some directions and too small for others. Adaptive methods use gradient history to rescale the step size automatically. This makes training less sensitive and often much faster at the beginning.

Q: Why is SGD your baseline?

Oral answer: SGD is the standard stochastic first-order method. It is simple, cheap per iteration and well understood theoretically. Therefore it is the natural baseline for checking whether adaptive methods really help.

Q: What does convexity mean?

Oral answer: Convexity means the loss landscape has no bad local minima. If we reach a stationary point in a convex problem, we have reached a global optimum. This makes convex logistic regression a clean baseline for comparing optimizers.

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$$

Q: Why is logistic loss convex?

Oral answer: Logistic loss has non-negative curvature everywhere. Its Hessian can be written as $X^\top DX$, where D has non-negative diagonal entries. Therefore the Hessian is positive semidefinite, so the loss is convex.

$$H = X^\top DX$$

For any vector z :

$$z^\top H z = z^\top X^\top D X z = (Xz)^\top D (Xz) \geq 0$$

because D has only non-negative diagonal entries.

Q: What does positive semidefinite mean?

Oral answer: A matrix is positive semidefinite if it has no negative curvature. Formally, $z^\top H z \geq 0$ for all vectors z . For optimization, this means the function is convex if the Hessian is positive semidefinite everywhere.

$$H \succeq 0 \iff z^\top H z \geq 0$$

2. Theory Basics

Q: What is the basic gradient descent update?

Oral answer: Gradient descent updates the parameter vector in the negative gradient direction. The gradient gives the direction of steepest increase, so we move in the opposite direction. The learning rate controls how large this movement is.

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

Here, x_t is the current point, $\nabla f(x_t)$ is the gradient, and η is the step size.

Q: What is the theoretical step-size choice?

Oral answer: For an L -smooth function, theory often requires a step size such as $\eta \leq 1/L$. This prevents the update from being too aggressive. In practice, however, L is often unknown or too conservative.

$$\eta \leq \frac{1}{L}$$

Here, L measures how quickly the gradient can change.

3. Non-Convex Regularization

Q: What makes your regularizer non-convex?

Oral answer: For small weights, the regularizer behaves similarly to an ℓ_2 penalty. For larger weights, it saturates instead of growing indefinitely. This curvature change introduces non-convexity.

$$R(w) = \lambda \sum_j \frac{\alpha w_j^2}{1 + \alpha w_j^2}$$

Q: What is the intuition behind this regularizer?

Oral answer: The term penalizes small and medium weights, but the penalty becomes bounded for large weights. So very large weights are not punished infinitely strongly. This creates a softer regularization effect compared to standard ℓ_2 -regularization.

$$\frac{\alpha w_j^2}{1 + \alpha w_j^2} \rightarrow 1 \quad \text{for large } |w_j|$$

5. SGD vs Adaptive Methods

Q: Why does the non-convex regularizer have limited impact?

Oral answer: At moderate λ , the regularization term is small compared to logistic loss. So the overall landscape is only mildly non-convex. That is why the optimizer ranking stays almost the same.

$$F(w) = \text{logistic loss} + \lambda R(w)$$

If λ is small, the second term only mildly changes the landscape.

Q: What happens if λ becomes larger?

Oral answer: If λ becomes larger, the regularizer starts to dominate. Then it pushes weights toward smaller values and can cause underfitting. In our λ -sweep, accuracy starts to drop when λ becomes too large.

Q: How is λ chosen?

Oral answer: λ is a hyperparameter chosen by the user. It controls the trade-off between fitting the data and regularizing the weights. In practice, we choose it using validation performance or a sweep over multiple values.

Q: Difference between SGD and adaptive methods?

Oral answer: SGD uses one global learning rate for all coordinates. Adaptive methods change the effective step size using past gradient information. This helps when coordinates have different scales or gradients vary strongly.

$$w_{t+1} = w_t - \eta g_t$$

Here, $g_t = \nabla f(w_t)$ is the gradient at iteration t . In SGD, the same η is used for every coordinate.

Q: What are typical assumptions for SGD theory?

Oral answer: Usually we assume smoothness, unbiased stochastic gradients and bounded variance. These assumptions allow us to prove that SGD makes progress in expectation. They are idealized, but useful for understanding the method.

$$\mathbb{E}[g_t] = \nabla f(w_t)$$

$$\mathbb{E}\|g_t - \nabla f(w_t)\|^2 \leq \sigma^2$$

4. Convergence Rates

Q: What does $O(1/\sqrt{T})$ mean intuitively?

Oral answer: It means convergence is sublinear. Progress continues, but becomes slower over time. To halve the error, we roughly need four times more iterations.

$$\text{error}(T) \sim \frac{1}{\sqrt{T}}$$

If we want:

$$\frac{1}{\sqrt{T_{\text{new}}}} = \frac{1}{2} \frac{1}{\sqrt{T}},$$

then:

$$T_{\text{new}} = 4T.$$

Q: Is $O(1/\sqrt{T})$ linear convergence?

Oral answer: No. Linear convergence means the error is multiplied by a constant factor in each step. $O(1/\sqrt{T})$ decreases much more slowly. That is why the curve eventually flattens.

$$\text{sublinear: } \text{error}(T) \sim \frac{1}{\sqrt{T}}$$

$$\text{linear: } \text{error}_t \leq c^t, \quad 0 < c < 1$$

Q: Difference between convergence rate and guarantee?

Oral answer: The rate says how fast the method converges. The guarantee says under which assumptions this rate is valid. For example, SGD needs assumptions like unbiased gradients and bounded variance.

Q: Why are empirical rates faster than theoretical rates?

Oral answer: Theoretical rates are worst-case guarantees. They are designed to hold for very general and sometimes adversarial functions. Our logistic and softmax objectives are nicer in practice, so observed convergence can be faster.

6. Adagrad

Q: How does Adagrad work?

Oral answer: Adagrad scales the update using accumulated past gradient information. Large accumulated gradients make the denominator larger and therefore reduce the effective step size. This makes the method adaptive instead of using one fixed learning rate.

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t} + \epsilon} g_t$$

where

$$G_t = G_{t-1} + g_t \cdot g_t$$

Formula explanation:

- w_t is the current parameter vector.
- $g_t = \nabla f(w_t)$ is the gradient at iteration t .
- η is the base learning rate.
- G_t stores accumulated squared gradient information.
- $g_t \cdot g_t = \|g_t\|^2$ measures the squared gradient magnitude.
- $\sqrt{G_t} + \epsilon$ is the adaptive denominator.
- ϵ avoids division by zero.

The effective learning rate is:

$$\eta_t^{\text{eff}} = \frac{\eta}{\sqrt{G_t} + \epsilon}$$

So if gradients have been large in the past, G_t becomes large and the next update becomes smaller.

Q: What is the intuition behind Adagrad?

Oral answer: Adagrad remembers how large previous gradients were. If the algorithm has already seen many large gradients, it becomes more cautious. This helps stabilize optimization and avoids taking too aggressive steps.

Main intuition:

$$G_t \uparrow \Rightarrow \sqrt{G_t} \uparrow \Rightarrow \eta_t^{\text{eff}} \downarrow$$

So the more gradient magnitude we accumulate, the smaller the effective step size becomes.

Q: Why can Adagrad slow down?

Oral answer: Adagrad accumulates gradient information forever. Therefore G_t keeps increasing over time. As a result, the effective learning rate keeps shrinking and training can become too conservative.

$$G_t = G_{t-1} + g_t \cdot g_t$$

Since $g_t \cdot g_t \geq 0$, we have:

$$G_t \geq G_{t-1}$$

Therefore:

$$\frac{\eta}{\sqrt{G_t} + \epsilon}$$

becomes smaller over time.

7. AdaGrad-Norm

Q: How does AdaGrad-Norm work?

Oral answer: AdaGrad-Norm uses one global scalar accumulator instead of adapting every coordinate separately. It accumulates the squared norm of the full gradient. Then it rescales the whole gradient update by this accumulated norm.

$$b_t^2 = b_{t-1}^2 + \|g_t\|^2$$

$$w_{t+1} = w_t - \frac{\eta}{b_t + \epsilon} g_t$$

Formula explanation:

- b_t is a scalar accumulator.
- $\|g_t\|^2$ is the squared norm of the full gradient vector.
- b_t^2 accumulates total gradient magnitude over time.
- $b_t + \epsilon$ is the adaptive denominator.
- The same scaling is applied to the whole gradient vector g_t .

The effective global learning rate is:

$$\eta_t^{\text{eff}} = \frac{\eta}{b_t + \epsilon}$$

So AdaGrad-Norm adapts the global step size based on the total gradient magnitude.

Q: Difference between Adagrad and AdaGrad-Norm?

Oral answer: Classical Adagrad is often written coordinate-wise, while our notation uses accumulated gradient magnitude. AdaGrad-Norm explicitly uses one shared scalar for the entire gradient vector. So AdaGrad-Norm is simpler and cleaner for theoretical analysis.

Adagrad: coordinate-wise or accumulated gradient scaling

AdaGrad-Norm: one global norm-based scaling

In AdaGrad-Norm, all coordinates are scaled by the same denominator $b_t + \epsilon$.

8. RMSprop

Q: How does RMSprop work?

Oral answer: RMSprop modifies Adagrad by using exponential forgetting. Instead of accumulating all squared gradients forever, it keeps a moving average of recent squared gradients. This prevents the learning rate from shrinking too aggressively.

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t$$

Formula explanation:

- v_t is the moving average of squared gradients.
- $\rho \in (0, 1)$ controls how much past information is kept.
- Large ρ , for example 0.9, means slow forgetting.
- $1 - \rho$ controls how strongly the current gradient affects v_t .
- g_t^2 is the squared gradient, usually taken element-wise.
- $\sqrt{v_t} + \epsilon$ rescales the update.

The effective learning rate is:

$$\eta_t^{\text{eff}} = \frac{\eta}{\sqrt{v_t} + \epsilon}$$

If recent gradients are large, v_t becomes large and the step size becomes smaller.

Q: Why is RMSprop different from Adagrad?

Oral answer: Adagrad remembers all past gradients forever. RMSprop forgets old gradients exponentially using ρ . Therefore RMSprop can keep adapting during long training.

Adagrad:

$$G_t = G_{t-1} + g_t^2$$

RMSprop:

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2$$

Main difference:

G_t only grows, but v_t can increase or decrease.

That is why RMSprop avoids the permanent learning-rate decay of Adagrad.

Q: Why can RMSprop diverge for large learning rates?

Oral answer: RMSprop is adaptive, but the base learning rate η still matters. If v_t becomes small, the denominator becomes small. Then the effective step size can become too large.

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t$$

If v_t is small:

$$\frac{\eta}{\sqrt{v_t} + \epsilon}$$

can become large.

So a large η can still cause unstable updates or divergence.

9. Adam

Q: How does Adam work?

Oral answer: Adam combines momentum with adaptive scaling. It first computes a moving average of gradients and a moving average of squared gradients. Then it uses bias correction and performs an adaptive update.

Step 1: first and second moment

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Step 2: bias correction

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Step 3: update

$$w_{t+1} = w_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Formula explanation:

- m_t is the first moment, meaning a moving average of gradients.
- v_t is the second moment, meaning a moving average of squared gradients.
- β_1 controls the memory of the gradient direction.
- β_2 controls the memory of the squared gradients.
- $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected versions.
- ϵ avoids division by zero.

Main intuition: Adam uses $\hat{m}_t$ for the update direction and $\hat{v}_t$ for adaptive step-size scaling.

Q: What is the first moment in Adam?

Oral answer: The first moment m_t is a moving average of gradients. It works similarly to momentum. It smooths the update direction and reduces noise.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

If gradients point in a similar direction over time, m_t builds up in that direction.

Q: What is the second moment in Adam?

Oral answer: The second moment v_t is a moving average of squared gradients. It measures how large gradients have recently been. Adam uses it in the denominator to scale the step size.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Large v_t means large recent gradients, so Adam reduces the effective step size.

Q: Why is bias correction needed in Adam?

Oral answer: Adam initializes $m_0 = 0$ and $v_0 = 0$. Therefore early moving averages are biased toward zero. Bias correction compensates for this underestimation.

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

At early iterations, $1 - \beta_1^t$ and $1 - \beta_2^t$ correct the fact that the averages have not yet accumulated enough history.

Q: Why can Adam fail to converge?

Oral answer: Adam uses exponential forgetting in the second moment. Therefore the denominator can sometimes decrease. This may make the effective learning rate unstable in pathological cases.

$$w_{t+1} = w_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

If $\hat{v}_t$ becomes too small, the effective step size can become too large. AMSGrad addresses this by forcing the denominator to be non-decreasing.

10. Results Interpretation

Q: Why do adaptive methods converge faster?

Oral answer: They rescale the update using gradient history. This helps avoid steps that are too large or too small in different directions. In our plots, this mainly improves the early loss decrease.

Q: Why are final accuracies similar?

Oral answer: The optimizers minimize the same objective. In the convex case, they should reach essentially the same solution if trained long enough. The main difference is the path and speed, not necessarily the final classifier.

Q: Does lower training loss always mean better test accuracy?

Oral answer: Not necessarily. Training loss measures optimization quality on the training data. Test accuracy measures generalization. In our results, adaptive methods reduce loss faster, but final test accuracy remains similar.

Q: What does the gradient norm plot tell us?

Oral answer: The gradient norm shows how close we are to a stationary point. Adaptive methods reduce large initial gradients faster because they normalize the update. This helps explain their faster early convergence.

$$\|\nabla f(x_t)\| \approx 0 \quad \Rightarrow \quad x_t \text{ is close to stationary.}$$

Q: Are your results trustworthy?

Oral answer: We average over 3 random seeds and show mean plus standard deviation. The trends are consistent across datasets and loss settings. For the final report, we plan to strengthen this with sensitivity and wall-clock analyses.

11. Experimental Setup

Q: What model are you using? Is it a neural network?

Oral answer: No, we use linear models. For A9a we use logistic regression, and for MNIST we use softmax regression. We chose this deliberately to isolate the optimizer effect.

Q: Why not neural networks?

Oral answer: Neural networks would introduce many additional factors, such as architecture, initialization and activations. Our goal is to compare optimizers in a controlled setting. Linear models make the optimization behavior easier to interpret.

Q: Why A9a and MNIST?

Oral answer: They represent two different regimes. A9a is sparse binary classification with 32K samples and 123 features. MNIST is higher-dimensional multi-class classification with 60K samples and 780 features after preprocessing.

Q: Why only 3 seeds?

Oral answer: For the in-class progress presentation, 3 seeds are enough to reduce obvious randomness and show stable trends. The curves were fairly consistent across runs. For the final report, we can increase the number of seeds if runtime allows.

Q: How did you choose learning rates?

Oral answer: We performed a log-scale learning-rate sweep. For each optimizer, we selected a learning rate that gave good final training loss. The sweep also showed that Adam is robust, while RMSprop can diverge for large η .

Q: Why mini-batches?

Oral answer: Mini-batches are the standard compromise between full-batch gradients and very noisy single-sample updates. They reduce gradient noise while keeping each update cheap. That matches the stochastic optimization setting of our methods.

Q: Did you implement everything from scratch?

Oral answer: Yes. The optimizers and loss functions were implemented from scratch in NumPy. This helps us understand exactly what each method is doing. It also avoids hiding optimizer behavior behind a deep learning framework.

Q: Do empirical results match the theory?

Oral answer: Qualitatively, yes. The theory suggests adaptive methods can improve convergence behavior, and empirically we see faster loss decrease. However, empirical rates are faster than worst-case bounds because our problems have more structure.

Q: Why care about rates if final accuracy is the same?

Oral answer: Compute budget matters. If one method reaches the same accuracy in 500 iterations and another needs 2000, that is a practical improvement. So convergence speed matters even when final performance is similar.

13. Outlook

Q: Why do a learning-rate sweep?

Oral answer: Optimizer performance can depend heavily on η . A sweep tells us how robust each method is to tuning. This is important because adaptive methods are supposed to reduce sensitivity.

Q: Why wall-clock time comparison?

Oral answer: Iteration count does not tell the whole story. Adaptive methods require extra computations per step. So we need to check whether they are still faster in real time.

Q: Why add L-BFGS?

Oral answer: L-BFGS is a quasi-Newton method from a different optimizer family. It gives us a useful reference beyond first-order adaptive methods. We plan to include it in the final report as a stronger classical baseline.

Q: Why not include L-BFGS already now?

Oral answer: For the in-class presentation, we focused on adaptive first-order methods because that is the core project topic. L-BFGS is planned as an additional reference baseline for the final report. So it is part of our outlook rather than the current main comparison.

14. Formula Intuition

Optimizer Formula Intuition

SGD:

$$w_{t+1} = w_t - \eta g_t$$

One fixed learning rate for the whole update.

Adagrad:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t, \quad G_t = G_{t-1} + g_t \cdot g_t$$

Accumulated gradient magnitude controls the step size.

AdaGrad-Norm:

$$b_t^2 = b_{t-1}^2 + \|g_t\|^2, \quad w_{t+1} = w_t - \frac{\eta}{b_t + \epsilon} g_t$$

One global adaptive learning rate based on the gradient norm.

RMSprop:

$$v_t = \rho v_{t-1} + (1 - \rho) g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t$$

Uses recent squared gradients and forgets old ones.

Adam:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$w_{t+1} = w_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Momentum plus adaptive scaling.

15. Quick-Fire Definitions

Fast Definitions

Positive definite: all eigenvalues > 0 ; strictly upward curvature.

Positive semidefinite: all eigenvalues ≥ 0 ; allows flat directions.

Lipschitz gradient: gradient does not change too abruptly.

Unbiased gradient: mini-batch gradient equals true gradient on average.

Momentum: running average of past gradients.

Bias correction: compensates Adam's zero initialization in early steps.

Sublinear convergence: error goes to zero slowly, e.g. $O(1/\sqrt{T})$.

Linear convergence: error decreases by a constant factor each step.

Superlinear convergence: faster than linear, e.g. Newton near optimum.

16. Emergency Answers

If you do not know exactly

Oral answer: We did not investigate this specific point in detail yet, but based on the theory I would expect ...

If they ask for more theory

Oral answer: The formal guarantee depends on assumptions like smoothness and bounded variance. Intuitively, the method works because it rescales the update based on gradient history.

If they criticize the results

Oral answer: That is a fair point. Our current results are progress results for the in-class presentation. For the final report, we plan to strengthen this with learning-rate sensitivity, wall-clock time and stronger non-convex regularization experiments.

Are adaptive methods always better?

Oral answer: No, not always. They are often faster early on, but they can require tuning and sometimes have weaker theory. Our results show they improve speed, but not necessarily final accuracy.

17. Ten Sentences to Memorize

Must-Know Sentences

1. Adaptive methods mainly improve optimization speed, not necessarily final accuracy.
2. We use linear models to isolate the optimizer effect.
3. Adagrad is strong because it adapts the step size using accumulated gradient magnitude.
4. RMSprop works well because it forgets old gradients and adapts quickly.
5. Adam combines momentum and adaptive scaling, but its theory is more delicate.
6. Final accuracies are similar because all methods optimize the same objective.
7. The non-convex regularizer has limited impact at moderate λ .
8. A9a and MNIST test two regimes: sparse binary and higher-dimensional multi-class.
9. Worst-case convergence bounds are conservative; empirical curves can be faster.
10. For the final report, we will add sensitivity analysis, wall-clock time, stronger λ and L-BFGS.

Ultra-Short Version for the Table

Main Message
Adaptive methods improve speed , not necessarily final accuracy.
Why Adaptive?
Fixed η is hard. Adaptive methods use gradient history to rescale updates.
SGD
Simple baseline, cheap, but needs careful η and has no coordinate adaptation.
Adagrad
Adaptive scaling based on accumulated gradient magnitude. Fast early progress, but can become conservative.
AdaGrad-Norm
One global scalar instead of per-coordinate scaling. Simpler, clean non-convex theory.
RMSprop
Forgets old gradients. Fast and practical, but weaker theory and can diverge at high η .

Adam
Momentum + adaptive scaling. Powerful, but convergence is more delicate.
A9a Result
RMSprop and Adagrad reduce loss fastest; all reach similar final accuracy.
MNIST Result
Adagrad fastest; adaptive scaling helps strongly in higher-dimensional settings.
Non-Convex Regularizer
Small λ = mild non-convexity, little effect. Large λ can dominate.
Why Linear Models?
Controlled setup. Isolates optimizer effect. Convex case gives clean theory.
Outlook
LR sweep, wall-clock time, stronger λ , L-BFGS baseline.